

MAPPA DEL PROGETTO FASTFOOD (PWM 2025/2026)

Stato della repo Innaxira05/ProgettoPWM24-25 al 09/10/2026

1. ALBERO DELLE CARTELLE

```
ProgettoPWM/
  README.md, .gitignore, PWM_project_25_26.pdf (traccia d'esame)
  frontend/  -> le pagine del sito (HTML + CSS + JS nel browser)
  backend/   -> il server Node.js + Express + MongoDB (API REST)
    config/   connessione al database
    data/     file JSON del professore (catalogo piatti)
    middleware/ funzioni che filtrano ogni richiesta (login, ruolo, errori)
    models/   struttura dei dati salvati in MongoDB
    routes/   le API, un file per ogni area del progetto
    scripts/  script lanciabili a mano (caricamento catalogo)
    utils/    funzioni di supporto riusabili
    node_modules/ librerie installate da npm (NON si modifica, NON va su GitHub)
```

2. BACKEND - FILE IN RADICE

```
server.js      Punto di ingresso. Si avvia con "npm run dev". Crea il server Express,
               collega le route a /api/..., attiva Swagger su /api-docs, serve la
               cartella frontend, si connette a MongoDB e, se il catalogo piatti e'
               vuoto, lo carica dal JSON.
swagger.js     Genera la documentazione OpenAPI leggendo i commenti @openapi scritti
               sopra ogni API in routes/. Richiesta dalla traccia.
package.json   Elenco delle librerie e degli script (start, dev, seed).
package-lock.json Versioni esatte delle librerie (generato da npm, non toccare).
.env           Configurazione segreta: MONGO_URI (Atlas), JWT_SECRET, tariffe
               consegna. NON va su GitHub (e' nel .gitignore).
README.md      Istruzioni di avvio e tabella degli endpoint.
_linkfonti.txt Appunto tuo: dove segnare le fonti (es. Bootstrap).
```

3. BACKEND - CARTELLE

```
config/db.js
  connectDB(): si collega a MongoDB usando MONGO_URI del .env.

data/
  meals 1.json   Il JSON del professore: 302 piatti, formato TheMealDB.
  LEGGIMI.txt    Nota su dove mettere il JSON.

middleware/
  auth.js        autenticato = controlla il token JWT nell'header Authorization.
                 soloRuolo('cliente' | 'ristoratore') = blocca chi ha il ruolo
                 sbagliato (errore 403). firmaToken = crea il JWT al login.
  validateId.js  Controlla che l'id nell'URL sia valido (altrimenti errore 400).
  errorHandler.js Trasforma ogni errore in una risposta JSON { message }.

models/ (ogni file = una collection di MongoDB)
  User.js        utenti: username, email, passwordHash, ruolo, dati personali,
                 pagamento (solo ultime 4 cifre), preferenze, allergie.
  Restaurant.js  ristoranti: owner (utente), nome, partita IVA, indirizzo,
                 telefono, coordinate GPS.
  Dish.js        piatti: UNA collection per due usi. ristorante = null -> catalogo
                 comune; ristorante = id -> piatto nel menu di quel ristorante.
  Order.js       ordini: cliente, ristorante, righe (con nome e prezzo copiati),
                 totali, modalita (ritiro/domicilio), stato.

routes/ (le API; il prefisso /api e' messo da server.js)
  auth.js        POST /auth/register, POST /auth/login
  users.js       GET, PUT, DELETE /users/me (profilo, modifica, elimina account)
  restaurants.js GET /restaurants (ricerca per nome, luogo, piatto), GET /:id,
                 GET e PUT /me, GET /me/stats (statistiche del ristorante)
  dishes.js      GET /dishes/catalog, GET /dishes (ricerche: nome, tipologia,
                 prezzo, ingrediente, allergeni), GET /:id,
                 POST, PUT, DELETE (solo ristorante, sul proprio menu)
  orders.js      GET /orders/stima-consegna, POST /orders, GET /orders,
                 GET /:id, PATCH /:id/avanza (il ristorante avanza lo stato)
  deliveries.js  GET /deliveries, PATCH /:id/ricevuto (il cliente conferma)
```

scripts/seed.js

"npm run seed": ricarica a mano il catalogo piatti dal JSON.

utils/

seedCatalog.js Legge il JSON e lo salva nella collection dishes. Contiene mappaPiatto(): QUI si adatta il formato del JSON.
allergens.js Ricava gli allergeni dagli ingredienti (parole chiave).
geo.js OpenStreetMap: indirizzo -> coordinate (Nominatim), distanza (OSRM), costo consegna = base + km x tariffa.
helpers.js Piccole funzioni: ricerca testo sicura, arrotondamento, ruoli.
HttpError.js Errore con codice HTTP (es. 404, 409).

4. FRONTEND

index.html Home: navbar, hero, 4 card dei macro-scenari, sezione "Piu ordinati" (card scritte a mano, ancora non lette dal backend), banner ristoranti, footer. Il solo JavaScript e' il menu hamburger mobile.
login.html Due moduli: login e registrazione (si cambia con ?mode=register). Valida i campi e chiama POST /api/auth/login e /api/auth/register. Dopo il login salva il token in localStorage con chiave "ff_token".
profilo.html Profilo in 3 tab: Dati, Modifica, Elimina account. Legge il token, chiama GET/PUT/DELETE su /users/me, ha il logout.
style.css Stile comune a tutte le pagine (variabili colori, navbar, hero, card, bottoni, footer, responsive).
profilo.css Stile solo della pagina profilo (tab, griglia dati, badge, form).
linkfonti.txt Appunto per le fonti (es. Bootstrap).

5. COME COMUNICANO LE PARTI

Browser (frontend) --fetch /api/...--> Express (routes) --> Mongoose (models)
--> MongoDB Atlas

1. Login: il backend controlla la password (hash bcrypt) e restituisce un JWT.
2. Il frontend salva il JWT in localStorage ("ff_token").
3. Ogni richiesta protetta invia "Authorization: Bearer <token>".
4. middleware/auth.js verifica il token e il ruolo, poi parte la route.
5. La route usa i models per leggere o scrivere su MongoDB.

Collection di MongoDB (database "fastfood"):
users, restaurants, dishes, orders

6. PUNTI DI ATTENZIONE (da sistemare, in ordine di importanza)

- 1) Nome del JSON. Il file si chiama "meals 1.json" (con spazio) ma il .env cerca data/meals_1.json. Soluzione: rinominare il file in meals_1.json oppure cambiare SEED_FILE nel .env.
- 2) Formato del JSON. Gli ingredienti sono negli array "ingredients" e "measures", non in strIngredient1, strIngredient2... Il seedCatalog.js attuale cerca questi ultimi, quindi caricherebbe i piatti SENZA ingredienti. Va corretto in mappaPiatto() prima di caricare il catalogo.
- 3) Il JSON non contiene allergeni ne prezzi: gli allergeni si ricavano dagli ingredienti, i prezzi li mette il ristorante.
- 4) Pagine linkate ma non ancora create: ristorante.html, ordini.html, consegne.html (i link in navbar e nelle card portano a pagine inesistenti).
- 5) profilo.html cerca un pulsante con id "btn-logout" che non esiste nell'HTML: in console compare un errore. Va aggiunto il pulsante o tolta la riga.
- 6) profilo.html usa l'indirizzo assoluto http://localhost:3000/api, login.html quello relativo /api: aprire sempre il sito da http://localhost:3000.
- 7) "Piu ordinati" in index.html e' statico: non legge ancora i piatti dal backend.
- 8) Il JSON e' presente due volte nella repo (radice e backend/data): tenere solo quello in backend/data.

7. CHECK-LIST PER L'ESAME (cosa copre la traccia, cosa manca)

Fatto lato backend (da testare): registrazione e login, modifica ed eliminazione utente, menu del ristorante, ricerca piatti e ristoranti, ordini con stato, consegna a domicilio con OpenStreetMap, statistiche, Swagger.

Da fare: pagine frontend mancanti, test di ogni API con Swagger, schermate di prova e relazione PDF finale, mail al professore con la composizione del gruppo.